



RUHR-UNIVERSITÄT BOCHUM

PARALLEL EXECUTION

Computer Architecture

Admin

- Next lecture on 9.1.2026
- No consultation hours until 19.2.2026

Agenda

- Flynn's Taxonomy
- Multiprocessor Topology
- Cache Coherency (MESI-Protocol)
- Coprocessors

Flynn's Taxonomy

More Parallelism

Parallelism so far: Pipelining, Tomasulo's Algorithm

Other possibilities:

- More processors?
- More data sets to work on?

We need a classification scheme for this

A Classification: Flynn's Taxonomy

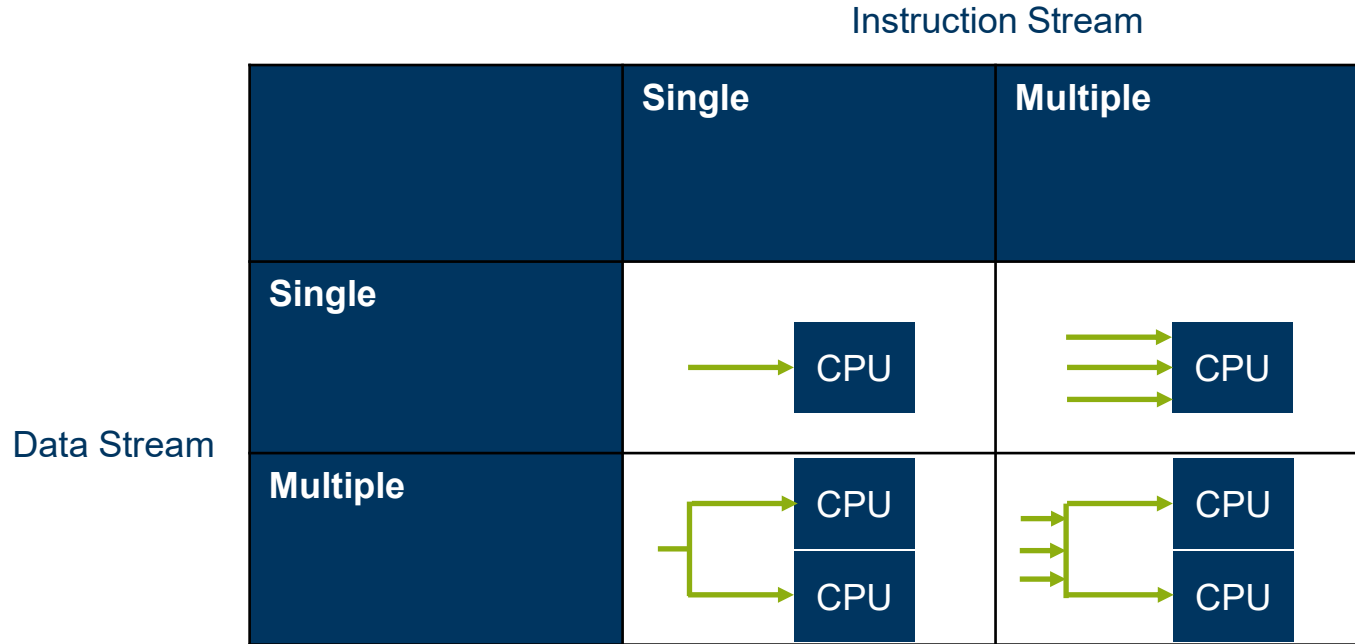
For our classification we either use multiple instruction inputs or multiple processing units



Here, we have a single instruction stream and a single processor that creates one output at a time and therefore a single data stream

After Flynn this is called **Single Instruction, Single Data Stream (SISD)**

Flynn's Taxonomy



SISD



Uniprocessors

Consists of a single CPU and instructions are executed sequentially

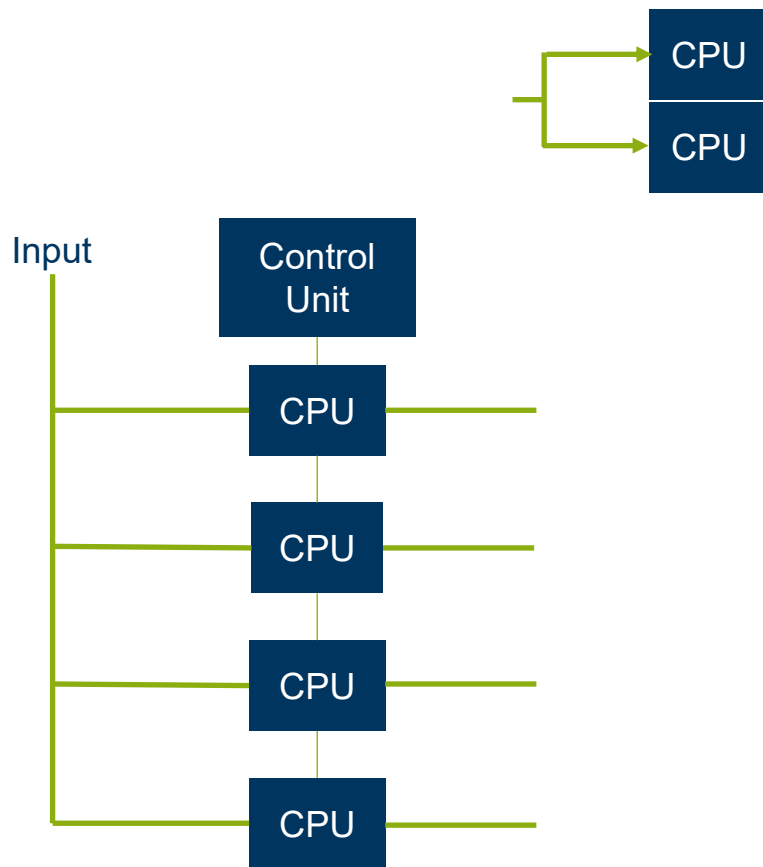
Can make use of pipelining and out of order execution

SIMD: Array Processors

Useful when data is highly parallelizable

Processors are all under the rule of one control unit

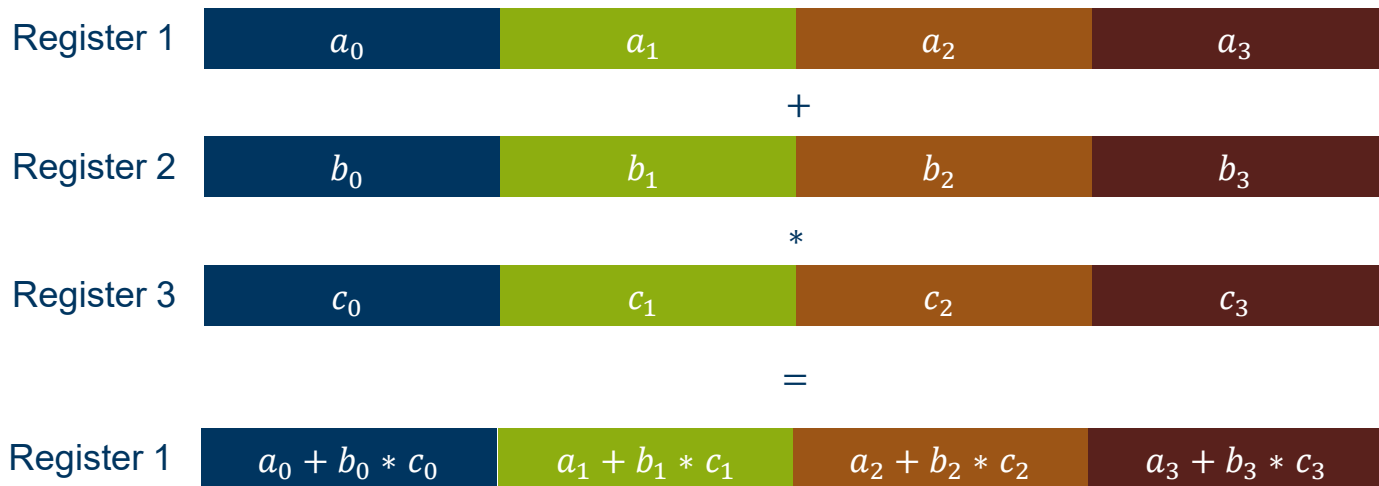
One instruction is executed on all processors on multiple data sets



SIMD Instructions

SIMD instruction sets contain extra instructions that are a combination of other instructions

Registers can have large length (128, 256,...)



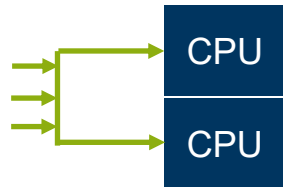
MISD



Not really used

Mostly: Multiple input streams into multiple processors, whose outputs are compared  Fault tolerance

MIMD



A set of processors simultaneously execute different instructions on different data sets

➡ Cluster, distributed/networked systems...

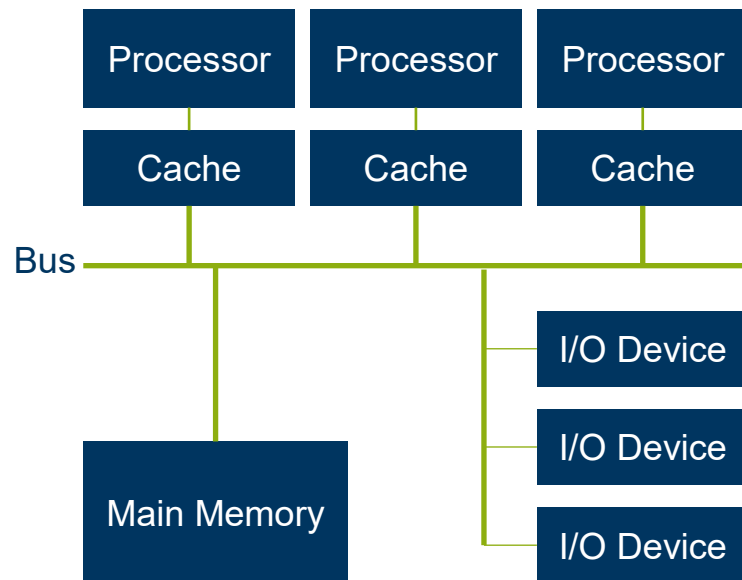
These processors can operate on shared memory and caches

Processors can also be entirely different systems

MIMD: Symmetric Multiprocessors

Characteristics:

- System consists of at least two processors of similar capability
- All processors share the main memory and I/O components
- All components are connected e.g. by a bus system
- All processors can perform the same functions
- The components are strongly coupled and all controlled by the same operating system



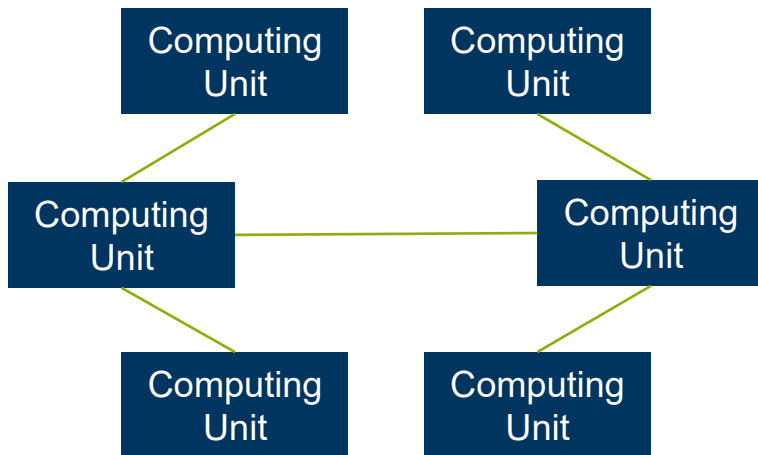
MIMD: Cluster

Loosely coupled connection of autonomous computing elements that appear as a single coherent system

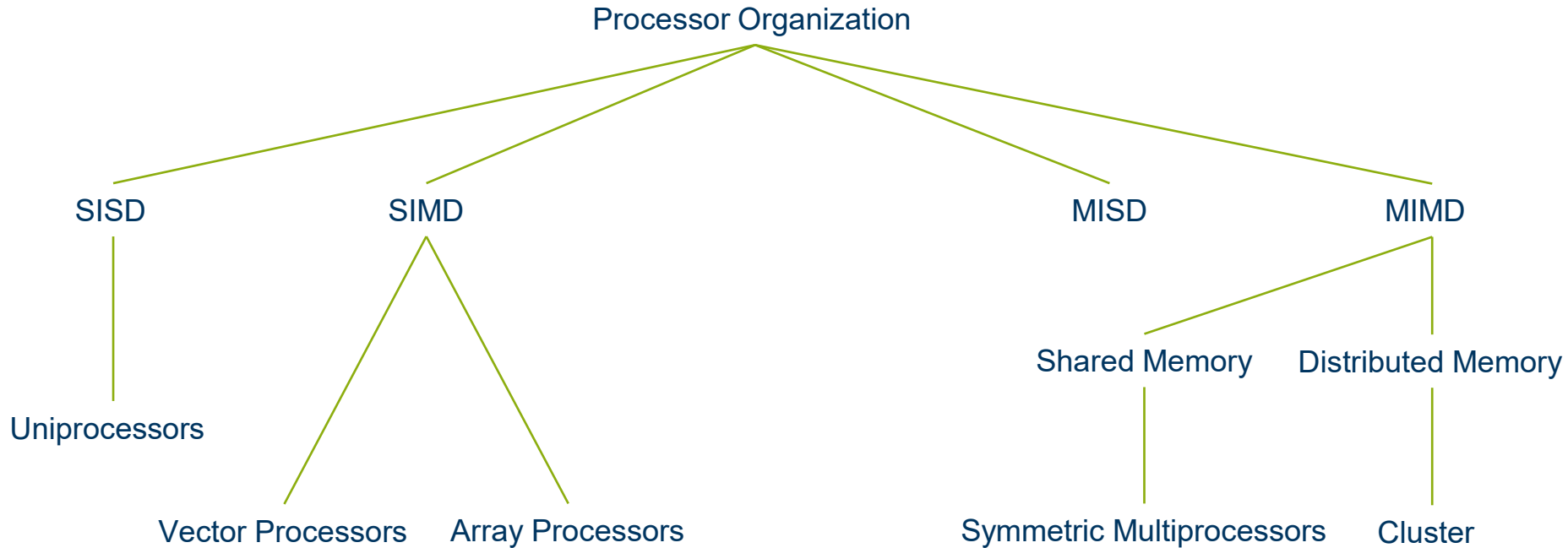
Usually no shared memory

No internal connection, network is used instead

Nodes can be connected in different manners

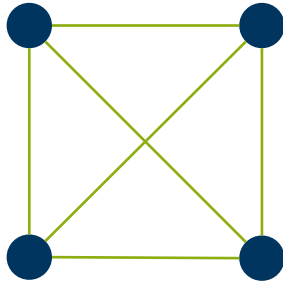


Overview



Multiprocessor Topology

Measures for connections of systems are cost and fault tolerance



Very fault tolerant but costly



Compromise



Very cheap but fault intolerant

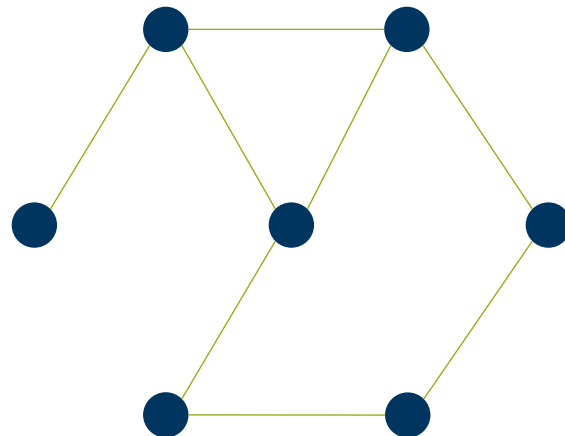
Multiprocessor Topology: Graph Structures

Topologies can be depicted with the use of graph structures:

$G = (V, E)$: Graph structure, where:

$V = \{1, \dots, n\}$: Number of processor nodes

$E \subseteq V \times V = \{\langle v_1, v_2 \rangle | v_1, v_2 \in V\}$:
Connections between nodes



Multiprocessor Topology: Graph Structures

Graph Structures have the following properties:

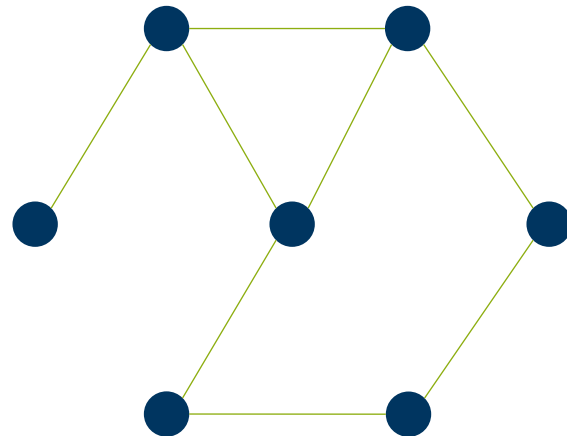
Diameter (G) =

$\max\{\forall a \forall b \mid \text{shortest path between } a \text{ and } b\}$

Degree (G) =

maximum number of edges a node participates in

$|\text{Connections}|(G) = |E|$



Diameter(G) = 4

Degree(G) = 3

$|\text{Connections}|(G) = 8$

Multiprocessor Topology: Star

Inner node works as a switch that passes data on between all other nodes

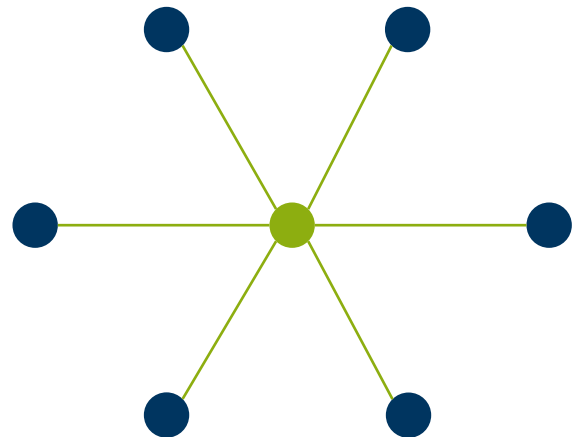
$\text{Diameter}(G) = 2$

$\text{Degree}(G) = |V|-1$

$|\text{Connections}|(G) = |V|-1$

Cheap, but if the inner node fails, the whole system fails

➡ Bottleneck problem



Multiprocessor Topology: Ring

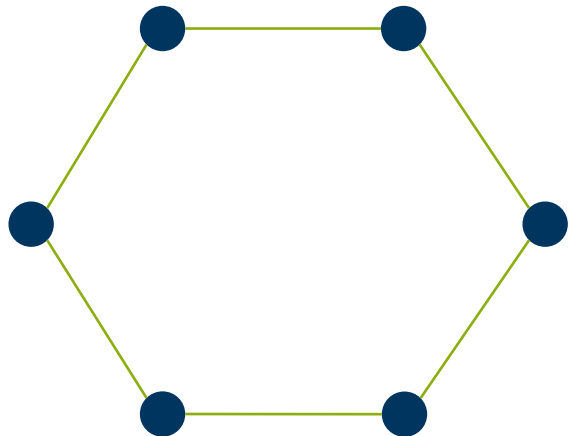
Each node is connected to its two neighbours and all connections form a ring

$\text{Diameter}(G) = \lfloor |V|/2 \rfloor$

$\text{Degree}(G) = 2$

$|\text{Connections}|(G) = |V|$

Cheap, and is able to withstand one failing node



Multiprocessor Topology: Mesh

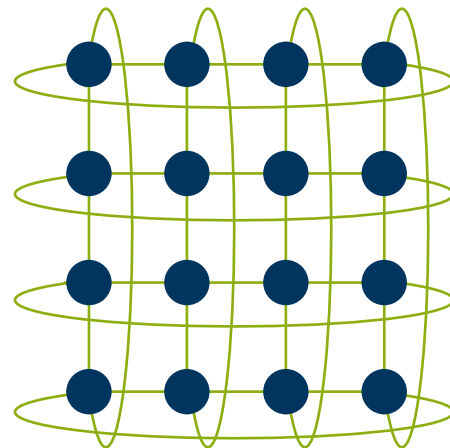
All nodes in the same row and all nodes in the same column form a ring

$$\text{Diameter}(G) = 2 \lceil |V|^{\frac{1}{2}} / 2 \rceil$$

$$\text{Degree}(G) = 4$$

$$|\text{Connections}|(G) = 2|V|$$

Fault tolerance is good while cost is moderately high



Multiprocessor Topology: Hypercube

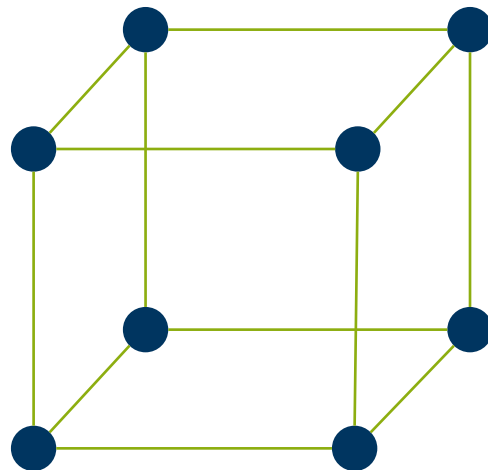
n-dimensional cube where $|V| = 2^n$

$$\text{Diameter}(G) = \log_2 |V|$$

$$\text{Degree}(G) = \log_2 |V|$$

$$|\text{Connections}|(G) = (|V| \log_2 |V|) / 2$$

Each n-cube can be converted into a (n+1)-cube by duplicating the previous shape and connecting all previous nodes with their duplicates

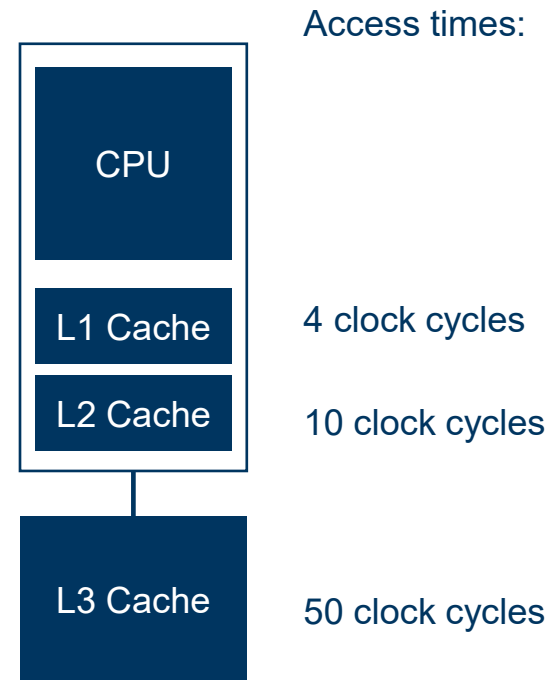


Recap: Cache Hierarchy

Sometimes multiple caches are used:

The fastest cache is accessed first

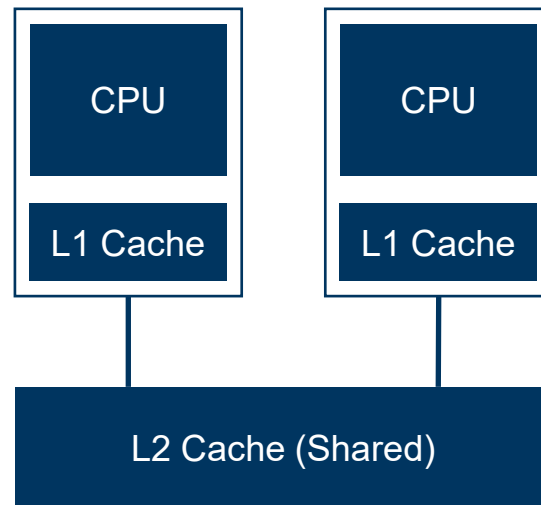
If a cache miss occurs the next fastest cache is checked



Cache Coherency

When multiple processors share a cache but also have individual caches the following needs to be ensured:

- Each processor only works on the newest computed value of a data entry
- The overhead of checking for newer values and updating them needs to be as low as possible



Cache Coherency: A Protocol

Read Accesses:

As long as only read accesses happen, no data entry needs to be updated

Write Accesses:

When a processor has exclusive access to a data entry no other processes need to be notified about writes

When a data entry is accessed by multiple processors and one of them updates it, all processors need to be notified. Can either update local copy or mark it as invalid.

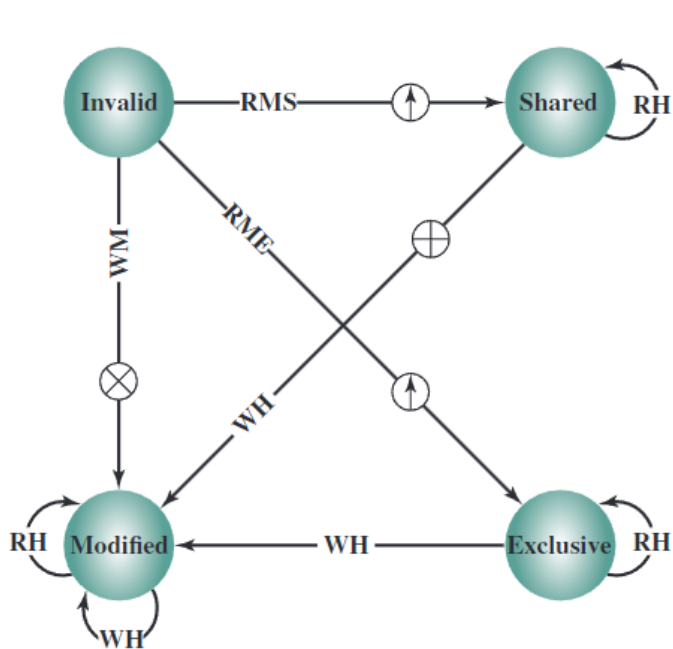
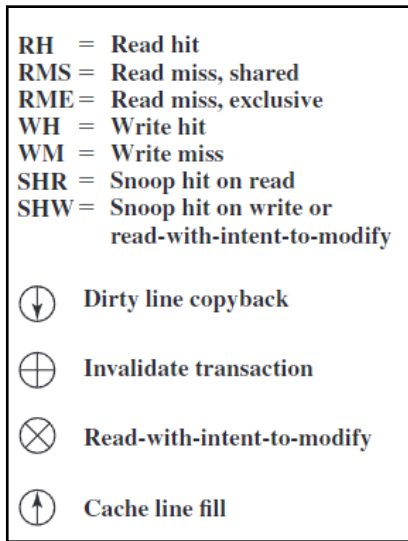
MESI-Protocol

Coherency protocol.

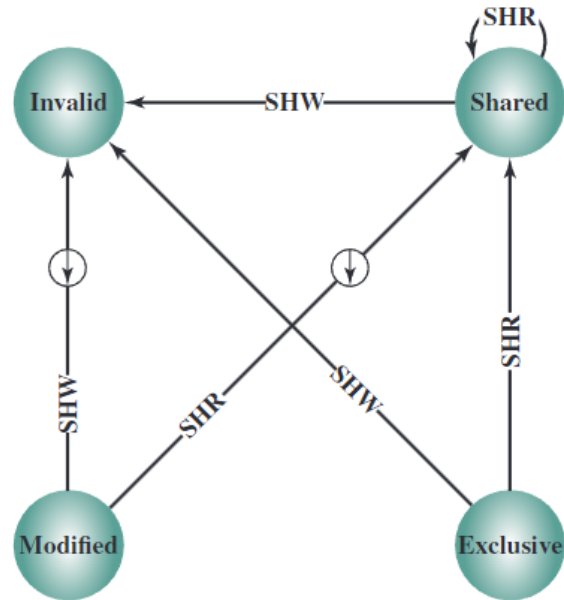
Consists of four states:

- **Modified:** The cache line has been modified by a processor and no copies exist in other processors
- **Exclusive:** The cache line has only been fetched by one processor and that copy is valid
- **Shared:** The cache line has been fetched by multiple processors and all copies are valid
- **Invalid:** The cache line is invalid

MESI-Protocol



(a) Line in cache at initiating processor

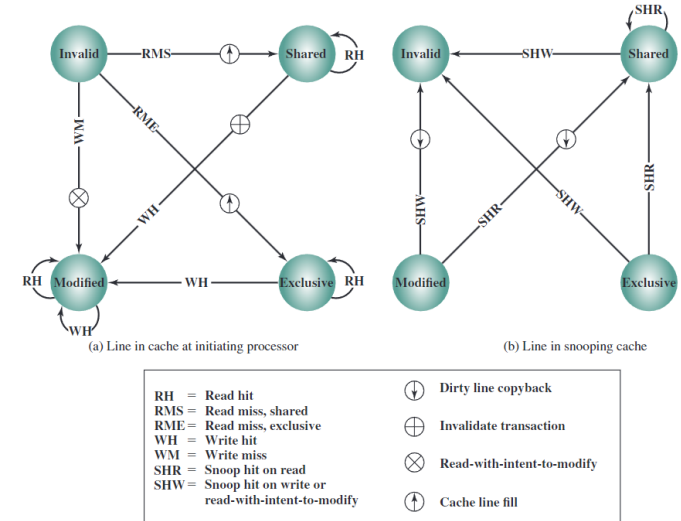


(b) Line in snooping cache

"MESI State Transition Diagram" from "Computer Organization and Architecture" by William Stallings 11. Edition, page 735

Example

Access	Processor 1	Processor 2
Initial	Invalid	Invalid
Processor 1 reads	Exclusive	Invalid
Processor 2 reads	Shared	Shared
Processor 1 writes	Modified	Invalid
Processor 2 reads	Shared	Shared



"MESI State Transition Diagram" from "Computer Organization and Architecture" by William Stallings 11. Edition, page 735

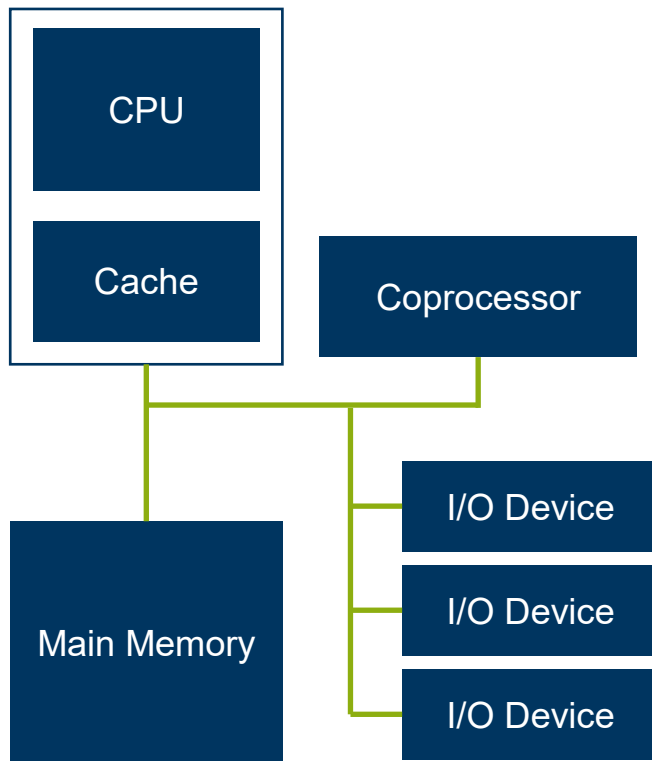
Coprocessors

Coprocessors

Specific purpose processors e.g. mathematics, graphics, cryptography or I/O processing

Perform tasks instead of the CPU so that it can perform other tasks

➡ Performance enhancement



Graphics Processing Unit (GPU)

Graphical Coprocessor

Operations used to display graphics on the screen (cartesian product, sine, cosine, scalar product...) are implemented directly into hardware

Contains thousands of cores that work together in batches

GPU: Example

Transform 3D scene into 2D representation on screen



Graphics Pipeline consists of three stages:

1. Application:

Apply changes made by user as they interact with the scene

2. Geometry:

Decide each objects place within the scene coordinates, transform it according to the cameras position, apply lightning and clip objects that reach out of the scene and scale the scene to the screen or window

3. Rasterization:

Map scene to screen's pixel and decide what each pixel is going to display

Summary

- Different types of parallelization can be categorized into four groups as per Flynn's Taxonomy
 - SISD: Unicomputers
 - SIMD: Array Processors
 - MISD: Not widely used
 - MIMD: Tightly coupled: Symmetric Multiprocessors / Loosely Coupled: Cluster
- In a cluster nodes can be connected according to different topology patterns
- Cache coherency for shared caches can be achieved using the MESI-Protocol
- Coprocessors are useful in enhancing the CPU's performance